🚀 Complete Beginner's Guide: Python Calculator with GitHub & Jenkins CI/CD

> NOTE
>
> **What you will build:** A Python calculator app, hosted on GitHub, automatically tested and deployed by Jenkins every time you push code. By the end, you'll have a working CI/CD pipeline you can demo in your presentation.

Table of Contents

1. Section 1 — Setup: Installing Everything
2. Section 2 — Python Fundamentals Used in This Project
3. Section 3 — The Calculator Code (Line-by-Line)
4. Section 4 — Git and GitHub
5. Section 5 — Jenkins: What Is CI/CD?
6. Section 6 — The Jenkinsfile (Line-by-Line)
7. Section 7 — Running and Verifying
8. Section 8 — Presentation Support

Section 1 — Setup: Installing Everything

1.0 — Which Operating System Do I Have?

Before we install anything, let's figure out your OS:

If you see...	You're on...
A **Start menu** and taskbar at the bottom	**Windows**
An **Apple logo** (🍎) in the top-left corner	**macOS**
A top bar that says **Activities** or a dock on the left	**Linux** (probably Ubuntu)

> **TIP**
>
> **On Windows:** Press `Windows key + R`, type `winver`, and press Enter. A window will pop up showing your Windows version (e.g., Windows 10 or 11).

1.1 — What Is a Terminal?

A **terminal** (also called **command prompt** or **shell**) is a text-based window where you type commands instead of clicking buttons. Think of it as texting your computer — you type an instruction, press Enter, and the computer does it.

How to open it:

- **Windows:** Press `Windows key`, type `cmd`, and click **Command Prompt**. (Alternatively, search for **PowerShell**.)
- **macOS:** Press `Cmd + Space`, type `Terminal`, press Enter.
- **Linux:** Press `Ctrl + Alt + T`.

You'll see a blinking cursor waiting for your input. That's normal — it's ready for you.

1.2 — Install Python

Python is a programming language — the language we'll write our calculator in.

Windows

1. Open your web browser and go to: **https://www.python.org/downloads/**
2. Click the big yellow button that says **"Download Python 3.x.x"** (the latest version).
3. **Run the downloaded file** (it will be named something like `python-3.12.x-amd64.exe`).
4. ⚠️ **IMPORTANT:** On the very first screen of the installer, check the box that says **"Add python.exe to PATH"** at the bottom. This lets you use Python from the terminal.
5. Click **"Install Now"** and wait for it to finish.
6. Click **Close**.

macOS

1. Go to **https://www.python.org/downloads/**
2. Download the macOS installer (`.pkg` file).
3. Double-click it and follow the prompts. Click **Continue** → **Install** → enter your password → **Close**.

Linux (Ubuntu/Debian)

Python usually comes pre-installed. Open a terminal and type:

```
bash
sudo apt update
sudo apt install python3 python3-pip -y
```

(`sudo` means "run this as an administrator"; `apt` is the tool that installs software on Ubuntu.)

Verify Python Is Installed

Open a terminal and type:

```
bash
python --version
```

What you should see (something like):

```
Python 3.12.4
```

If that doesn't work (especially on macOS/Linux), try:

```
bash
python3 --version
```

> NOTE
>
> Throughout this guide, if `python` doesn't work for you, replace it with `python3` everywhere. Same goes for `pip` → `pip3` .

1.3 — Install Git

Git is a version control tool — it tracks every change you make to your code, like an "undo history" on steroids. We'll use it to save our code and push it to GitHub.

Windows

1. Go to **https://git-scm.com/downloads/win**
2. Download the installer and run it.
3. Click **Next** through all screens — the default settings are fine.
4. Click **Install**, then **Finish**.

macOS

Open Terminal and type:

```
bash
git --version
```

If Git isn't installed, macOS will prompt you to install **Xcode Command Line Tools**. Click **Install** and wait.

Linux (Ubuntu/Debian)

```
bash
sudo apt install git -y
```

Verify Git Is Installed

```
bash
git --version
```

What you should see:

```
git version 2.43.0
```

(The exact number may differ — that's fine.)

1.4 — Create a GitHub Account

GitHub is a website that stores your Git repositories (project folders) online so others (and Jenkins) can access them.

1. Go to **https://github.com** in your browser.
2. Click **Sign up**.
3. Enter your email, create a password, and choose a username.
4. Complete the verification puzzle and click **Create account**.
5. Check your email and enter the verification code GitHub sends you.
6. You now have a GitHub account. **Remember your username** — you'll need it later.

1.5 — Install Java (Required for Jenkins)

Jenkins is written in Java, so Java must be installed first.

Windows

1. Go to **https://adoptium.net/** (the official place to get Java).
2. Click the big **"Latest LTS Release"** button to download the installer.
3. Run the installer. On the **Custom Setup** screen, make sure **"Set JAVA_HOME variable"** is enabled (click the ✕ next to it and select "Will be installed on local hard drive").
4. Click **Next** → **Install** → **Finish**.

macOS

```
bash
brew install openjdk@17
```

(If you don't have `brew`, install it first from https://brew.sh by pasting the command shown on that page into Terminal.)

Linux

```
bash
sudo apt install openjdk-17-jdk -y
```

Verify Java Is Installed

```
bash
java -version
```

What you should see (something like):

```
openjdk version "17.0.8" 2023-07-18
```

1.6 — Install Jenkins

Jenkins is an automation server — it will automatically test and build our calculator every time we push code to GitHub.

Windows

1. Go to **https://www.jenkins.io/download/**
2. Under **Long Term Support (LTS)**, click **Windows**.
3. Run the downloaded `.msi` installer.
4. Click **Next** through the screens. When it asks about the **service logon type**, choose **"Run service as LocalSystem"** (the easiest option).
5. It will ask for an **HTTP port** — leave it as **8080** (the default).
6. The installer will detect your Java installation. Click **Next** → **Install** → **Finish**.
7. Jenkins will start automatically as a Windows service.

macOS

```
bash
brew install jenkins-lts
brew services start jenkins-lts
```

Linux (Ubuntu/Debian)

```
bash
# Add Jenkins repository key and source
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-
stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | sudo tee
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins -y

# Start Jenkins
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Verify Jenkins Is Running

1. Open your web browser.
2. Go to **http://localhost:8080**
3. You should see a page that says **"Unlock Jenkins"** with a message asking for an **administrator password**.

This means Jenkins is installed and running! Don't close this page — we'll unlock it in Section 5.

> IMPORTANT
>
> If the page doesn't load, wait 1–2 minutes. Jenkins takes a moment to start up. If it still doesn't load, make sure the Jenkins service is running.

Section 2 — Python Fundamentals Used in This Project

Before we write the calculator, let's understand the Python building blocks we'll use. You don't need to memorize these — just read through so the code makes sense when you see it.

2.1 — Variables

A **variable** is a named box that stores a value.

```
python
name = "Varun"       # stores the text "Varun"
age = 20             # stores the number 20
price = 9.99         # stores a decimal number
```

2.2 — Functions

A **function** is a reusable block of code that performs a specific task. Think of it as a recipe — you define it once, then call it whenever you need it.

```
python
def greet(name):           # 'def' means "define a function"; 'name' is the input
    return "Hello, " + name  # 'return' sends a result back

message = greet("Varun")   # calling the function; message becomes "Hello, Varun"
```

- `def` — keyword that starts a function definition
- `name` — a **parameter** (input the function expects)
- `return` — sends a value back to whoever called the function

2.3 — Conditionals (if / elif / else)

Conditionals let your code make decisions.

```
python
choice = "2"

if choice == "1":
    print("You picked addition")
elif choice == "2":
    print("You picked subtraction")  # ← This runs because choice is "2"
else:
    print("Invalid choice")
```

- `if` — checks the first condition
- `elif` — ("else if") checks another condition if the previous one was false
- `else` — runs if none of the above conditions were true
- `==` — means "is equal to" (not `=`, which sets a value)

2.4 — Input and Output

```
python
name = input("What is your name? ")  # Shows the question, waits for user to type
print("Hello, " + name)              # Displays text on screen
```

- `input()` — pauses the program and waits for the user to type something; always returns **text** (a string)
- `print()` — displays a message on screen

2.5 — Type Conversion

Since `input()` always returns text, we need to convert it to a number for math:

```
python
text = input("Enter a number: ")  # User types "5" → text is the string "5"
number = float(text)               # Converts "5" to the number 5.0
```

- `float()` — converts text to a decimal number (e.g., "3.14" → 3.14)
- If the text isn't a valid number (e.g., "hello"), `float()` will crash — we handle that with `try/except`.

2.6 — Error Handling (try / except)

```
python
try:
    number = float("hello")  # This will fail
except ValueError:
    print("That's not a valid number!")  # This runs instead of crashing
```

- `try` — "try running this code"
- `except ValueError` — "if a ValueError occurs, run this instead of crashing"

2.7 — Modules and Importing

A **module** is just a `.py` file. You can use code from one file in another file using `import`.

```
python
# In file calculator.py:
def add(a, b):
    return a + b

# In file test_calculator.py:
from calculator import add   # brings the 'add' function into this file
result = add(2, 3)            # result is 5
```

2.8 — Unit Testing with `unittest`

Unit testing means writing code that automatically checks if your code works correctly. Python has a built-in module called `unittest` for this.

```
python
import unittest                    # import the testing module

class TestMath(unittest.TestCase): # a group of related tests
    def test_addition(self):       # each test is a function starting with 'test_'
        self.assertEqual(2 + 2, 4) # check that 2+2 equals 4
```

- `unittest.TestCase` — a class (blueprint) that gives us testing tools
- `self.assertEqual(a, b)` — checks that `a` equals `b`; if not, the test **fails**

Section 3 — The Calculator Code (Line-by-Line)

Now let's create the actual project files. We need three files:

File	Purpose
`calculator.py`	The calculator logic (functions for add, subtract, etc.)
`test_calculator.py`	Automated tests that verify the calculator works
`Jenkinsfile`	Instructions for Jenkins (covered in Section 6)

3.1 — Create the Project Folder

Open your terminal and run these commands:

Windows (Command Prompt):

```
cmd
cd %USERPROFILE%\Desktop
mkdir Varun_Calculator
cd Varun_Calculator
```

macOS / Linux:

```
bash
cd ~/Desktop
mkdir Varun_Calculator
cd Varun_Calculator
```

What this does:

- `cd` — "change directory" (move into a folder)
- `mkdir` — "make directory" (create a new folder)
- You're now inside a folder called `Varun_Calculator` on your Desktop.

3.2 — `calculator.py` — The Calculator Application

Create a file called `calculator.py` inside the `Varun_Calculator` folder. You can do this by:

- **Option A:** Open Notepad (Windows) or TextEdit (Mac), paste the code below, and save as `calculator.py` in the `Varun_Calculator` folder. Make sure the file type is **"All Files"**, not `.txt`.
- **Option B:** If you have VS Code installed, open the folder and create the file there.

Here's the complete code:

```python
# calculator.py
# A simple calculator that performs basic arithmetic operations.

# ----- CALCULATOR FUNCTIONS -----

def add(a, b):
    """Return the sum of a and b."""
    return a + b


def subtract(a, b):
    """Return the difference of a and b."""
    return a - b


def multiply(a, b):
    """Return the product of a and b."""
    return a * b


def divide(a, b):
    """Return the result of a divided by b.
    Raises a ValueError if b is zero (division by zero is undefined).
    """
    if b == 0:
        raise ValueError("Cannot divide by zero!")
    return a / b


# ----- MAIN PROGRAM (runs when you execute this file directly) -----

def main():
    """Display a menu, take user input, and perform the chosen operation."""
    print("============================")
    print("   SIMPLE CALCULATOR")
    print("============================")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("============================")

    choice = input("Enter your choice (1/2/3/4): ")

    # Validate the menu choice
    if choice not in ("1", "2", "3", "4"):
```

```
        print("Error: Invalid choice. Please enter 1, 2, 3, or 4.")
        return  # exit the function early

    # Get numbers from the user
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
    except ValueError:
        print("Error: Please enter valid numbers.")
        return

    # Perform the chosen operation
    if choice == "1":
        result = add(num1, num2)
        operation = "+"
    elif choice == "2":
        result = subtract(num1, num2)
        operation = "-"
    elif choice == "3":
        result = multiply(num1, num2)
        operation = "*"
    elif choice == "4":
        try:
            result = divide(num1, num2)
        except ValueError as e:
            print(f"Error: {e}")
            return
        operation = "/"

    print(f"\nResult: {num1} {operation} {num2} = {result}")


# This block runs only when you execute 'python calculator.py' directly.
# It does NOT run when another file imports this file.
if __name__ == "__main__":
    main()
```

Line-by-Line Explanation

Lines	What it does
`def add(a, b):`	Defines a function named `add` that takes two inputs `a` and `b`.
`return a + b`	Sends back the result of adding `a` and `b`.
`def subtract(a, b):`	Same pattern — subtracts `b` from `a`.
`def multiply(a, b):`	Multiplies `a` and `b`.
`def divide(a, b):`	Divides `a` by `b`, but first checks for zero.
`if b == 0:`	If someone tries to divide by zero...
`raise ValueError(...)`	...we deliberately trigger an error with a message, instead of letting Python crash with a confusing error.
`def main():`	The main function that ties everything together — shows the menu, gets input, calls the right math function.
`choice = input(...)`	Asks the user to pick an operation (1–4).
`if choice not in ("1","2","3","4"):`	Checks if the user typed something other than 1, 2, 3, or 4.
`float(input(...))`	Asks for a number and converts the text input to a decimal number.
`except ValueError:`	If the user typed letters instead of a number, catch the error gracefully.
`f"Result: {num1} {operation} {num2} = {result}"`	An **f-string** — lets you embed variables inside text using `{}`.
`if __name__ == "__main__":`	A Python idiom: run `main()` only when this file is executed directly (not when it's imported by the test file).

Try It Out

In your terminal (make sure you're in the `Varun_Calculator` folder):

```
bash
python calculator.py
```

What you should see:

```
============================
   SIMPLE CALCULATOR
============================
1. Addition
2. Subtraction
3. Multiplication
4. Division
============================
Enter your choice (1/2/3/4):
```

Type `1`, press Enter. Then type `10`, Enter. Then `5`, Enter. You should see:

```
Result: 10.0 + 5.0 = 15.0
```

3.3 — `test_calculator.py` — The Automated Tests

Create a second file called `test_calculator.py` in the same folder:

```
python
# test_calculator.py
# Automated tests for the calculator functions.
# These tests run automatically to verify our calculator works correctly.

import unittest
from calculator import add, subtract, multiply, divide


class TestCalculator(unittest.TestCase):
    """A collection of tests for each calculator function."""

    # ----- ADDITION TESTS -----

    def test_add_positive_numbers(self):
        """Test adding two positive numbers."""
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        """Test adding two negative numbers."""
        self.assertEqual(add(-1, -1), -2)

    def test_add_zero(self):
        """Test adding zero to a number."""
        self.assertEqual(add(5, 0), 5)

    # ----- SUBTRACTION TESTS -----

    def test_subtract_positive_numbers(self):
        """Test subtracting two positive numbers."""
        self.assertEqual(subtract(10, 4), 6)

    def test_subtract_negative_result(self):
        """Test subtraction that results in a negative number."""
        self.assertEqual(subtract(3, 7), -4)

    # ----- MULTIPLICATION TESTS -----

    def test_multiply_positive_numbers(self):
        """Test multiplying two positive numbers."""
        self.assertEqual(multiply(3, 4), 12)

    def test_multiply_by_zero(self):
        """Test multiplying by zero (result should always be 0)."""
        self.assertEqual(multiply(100, 0), 0)

    # ----- DIVISION TESTS -----
```

```
    def test_divide_positive_numbers(self):
        """Test dividing two positive numbers."""
        self.assertEqual(divide(10, 2), 5)

    def test_divide_returns_float(self):
        """Test that division returns a decimal result when needed."""
        self.assertEqual(divide(7, 2), 3.5)

    def test_divide_by_zero(self):
        """Test that dividing by zero raises a ValueError."""
        with self.assertRaises(ValueError):
            divide(10, 0)


# This allows running the tests by executing: python test_calculator.py
if __name__ == "__main__":
    unittest.main()
```

Line-by-Line Explanation

Lines	What it does
`import unittest`	Loads Python's built-in testing framework.
`from calculator import add, subtract, multiply, divide`	Imports our four functions from `calculator.py` so we can test them.
`class TestCalculator(unittest.TestCase):`	Creates a test class — a container for related tests. It inherits from `unittest.TestCase` to get testing superpowers.
`def test_add_positive_numbers(self):`	Defines one test. The name **must** start with `test_` so Python knows it's a test.
`self.assertEqual(add(2, 3), 5)`	Calls `add(2, 3)` and checks if the result equals `5`. If not, the test **fails**.
`with self.assertRaises(ValueError):`	Checks that the code inside raises a `ValueError`. We use this to verify that `divide(10, 0)` correctly rejects division by zero.
`if __name__ == "__main__": unittest.main()`	If you run this file directly, it will execute all the tests.

Run the Tests Locally

```
bash
python -m unittest test_calculator.py -v
```

(`-m unittest` means "run the unittest module"; `-v` means "verbose" — show each test's name and result.)

What you should see:

```
test_add_negative_numbers (test_calculator.TestCalculator) ... ok
test_add_positive_numbers (test_calculator.TestCalculator) ... ok
test_add_zero (test_calculator.TestCalculator) ... ok
test_divide_by_zero (test_calculator.TestCalculator) ... ok
test_divide_positive_numbers (test_calculator.TestCalculator) ... ok
test_divide_returns_float (test_calculator.TestCalculator) ... ok
test_multiply_by_zero (test_calculator.TestCalculator) ... ok
test_multiply_positive_numbers (test_calculator.TestCalculator) ... ok
test_subtract_negative_result (test_calculator.TestCalculator) ... ok
test_subtract_positive_numbers (test_calculator.TestCalculator) ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.001s

OK
```

✅ All 10 tests passed! Every `ok` means that test checked out correctly.

Section 4 — Git and GitHub

4.1 — What Is Version Control?

Imagine you're writing an essay. You save it as `essay_v1.docx`, then `essay_v2.docx`, then `essay_final.docx`, then `essay_final_FINAL.docx` ... Version control solves this problem. **Git** tracks every change you make and lets you go back to any previous version. It's like having an unlimited undo history for your entire project.

Key vocabulary:

Term	Meaning
Repository (repo)	A project folder tracked by Git
Commit	A snapshot of your project at a point in time (like "Save" with a description)
Stage	Marking files to be included in the next commit
Push	Uploading your commits from your computer to GitHub
Clone	Downloading a repository from GitHub to your computer
Branch	A parallel copy of your code where you can experiment without affecting the main version

4.2 — Configure Git (One-Time Setup)

Tell Git who you are (this info appears in your commits):

```
bash
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Replace with your actual name and the email you used for GitHub.

4.3 — Create a Repository on GitHub

1. Go to **https://github.com** and log in.
2. Click the **+** icon in the top-right corner → **New repository**.
3. Fill in:
    - **Repository name:** `Varun_Calculator`
    - **Description:** `Simple Python Calculator with CI/CD Pipeline`
    - **Visibility:** Public (so Jenkins can access it easily)
    - ❌ Do **NOT** check "Add a README file" (we'll push our own files)
4. Click **Create repository**.
5. You'll see a page with setup instructions. **Keep this page open** — you'll need the URL shown (it looks like `https://github.com/YOUR_USERNAME/Varun_Calculator.git`).

4.4 — Initialize Git and Push Your Code

Open your terminal. Make sure you're inside the `Varun_Calculator` folder:

```
bash
cd %USERPROFILE%\Desktop\Varun_Calculator
```

(On macOS/Linux: `cd ~/Desktop/Varun_Calculator`)

Now run these commands **one at a time**:

```
bash
git init
```

What it does: Creates a hidden `.git` folder that turns this folder into a Git repository. **What you should see:** `Initialized empty Git repository in .../Varun_Calculator/.git/`

```
bash
git add .
```

What it does: Stages ("marks") all files in the folder to be included in the next commit. The `.` means "everything in the current folder."

```
bash
git commit -m "Initial commit: add calculator and tests"
```

What it does: Takes a snapshot of all staged files. The `-m` flag lets you add a message describing what this commit contains. **What you should see:** Something like `2 files changed, 95 insertions(+)`.

```
bash
git branch -M main
```

What it does: Renames the default branch to `main` (the standard name used by GitHub).

```
bash
git remote add origin https://github.com/YOUR_USERNAME/Varun_Calculator.git
```

What it does: Tells Git where to push your code online. Replace `YOUR_USERNAME` with your actual GitHub username.

- `origin` — a shorthand name for the remote URL (like a nickname).

```
bash
git push -u origin main
```

What it does: Uploads your commits to GitHub. The `-u` flag sets this as the default destination so future pushes just need `git push`.

What you should see: A progress bar, then a message ending with `Branch 'main' set up to track remote branch 'main' from 'origin'.`

> NOTE
>
> **Authentication:** GitHub will ask for your credentials. It no longer accepts passwords — you need a **Personal Access Token**:
>
> 1. Go to GitHub → click your profile picture → **Settings** → **Developer settings** (very bottom of left sidebar) → **Personal access tokens** → **Tokens (classic)**.
> 2. Click **Generate new token (classic)**.
> 3. Give it a name (e.g., "Calculator Project"), set expiration to 90 days, and check the **`repo`** scope (full control of repositories).
> 4. Click **Generate token** and **copy the token immediately** (you won't see it again).
> 5. When Git asks for your password, paste this token instead.

Verify on GitHub

Go to `https://github.com/YOUR_USERNAME/Varun_Calculator` in your browser. You should see `calculator.py` and `test_calculator.py` listed there. 🎉

4.5 — Branches (Brief Explanation)

A **branch** is like a parallel universe for your code. The `main` branch is your "official" version. You can create other branches to experiment:

```
bash
git checkout -b feature-new-operation   # creates and switches to a new branch
# ... make changes ...
git checkout main                        # switch back to main
```

Why branches matter for CI/CD: In a team, developers work on separate branches. Jenkins can test every branch, and code only goes to `main` after all tests pass. For this project, we'll work on `main` for simplicity.

Section 5 — Jenkins: What Is CI/CD?

5.1 — CI/CD in Plain Language

Term	Stands for	What it means
CI	Continuous Integration	Every time you push code, it's automatically tested to catch bugs early
CD	Continuous Delivery / Deployment	After tests pass, the code is automatically packaged and delivered (deployed)

Without CI/CD: You write code → manually run tests → manually copy files to a server → hope nothing breaks.

With CI/CD: You push code → Jenkins automatically pulls it → runs tests → builds it → deploys it. All hands-free. If something breaks, Jenkins tells you immediately.

The pipeline flow:

```
You push code → GitHub → Jenkins detects change → Checkout → Test → Build → Deploy
```

5.2 — Unlock Jenkins

1. Open your browser and go to **http://localhost:8080**
2. Jenkins shows you a page saying **"Unlock Jenkins"** and tells you where to find the password.

Find the initial admin password:

- **Windows:** Open File Explorer and navigate to:

  ```
  C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
  ```

  Or open Command Prompt and type:

  ```
  cmd
  type "C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword"
  ```

- **macOS:**

```
bash
cat /Users/$(whoami)/.jenkins/secrets/initialAdminPassword
```

- **Linux:**

```
bash
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

3. **Copy the long string of characters** that appears and **paste** it into the Jenkins unlock page.
4. Click **Continue**.

> **TIP**
>
> The password is a random string like `a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6`. Make sure you copy the entire thing with no extra spaces.

5.3 — Initial Jenkins Setup

1. On the next screen, click **"Install suggested plugins"** — Jenkins will download and install the most commonly used plugins. This takes a few minutes. You'll see a progress screen with green checkmarks.
2. After plugins are installed, **create your admin user**:
   - Username: `admin` (or whatever you like)
   - Password: choose something you'll remember
   - Full name: your name
   - Email: your email
3. Click **Save and Continue**.
4. On the **Instance Configuration** page, leave the URL as `http://localhost:8080/` and click **Save and Finish**.
5. Click **"Start using Jenkins"**.

You should now see the **Jenkins Dashboard** — a page with "Welcome to Jenkins!" at the top. 🎉

5.4 — Configure Python in Jenkins

Jenkins needs to know where Python is installed.

Find Your Python Path

Windows:

```
cmd
where python
```

You'll see something like: `C:\Users\sriva\AppData\Local\Programs\Python\Python312\python.exe`

macOS / Linux:

```
bash
which python3
```

You'll see something like: `/usr/bin/python3`

Keep this path noted — we'll use it in the Jenkinsfile.

> IMPORTANT
>
> On Windows, the Jenkins service runs as a different user. To ensure Jenkins can find Python, we use the full path to `python.exe` in the Jenkinsfile, or make sure Python is in the system PATH for all users.

5.5 — Install Pipeline Plugin (If Not Already Installed)

The "Install suggested plugins" step usually includes the Pipeline plugin, but let's verify:

1. In Jenkins, click **Manage Jenkins** (left sidebar) → **Plugins** → **Installed plugins** tab.
2. Search for **"Pipeline"** — if you see it listed, you're good.
3. If not, click the **Available plugins** tab, search for **"Pipeline"**, check it, and click **Install**.

5.6 — Create a Pipeline Job

1. On the Jenkins Dashboard, click **"New Item"** (left sidebar).
2. Enter the name: `Calculator-Pipeline`
3. Select **"Pipeline"** (not "Freestyle project").
4. Click **OK**.
5. You'll be taken to the configuration page. Scroll down to the **"Pipeline"** section.
6. In the **Definition** dropdown, select **"Pipeline script from SCM"**.
    - **SCM** stands for "Source Code Management" — this tells Jenkins to look for the Jenkinsfile in your GitHub repo.
7. Select **Git** as the SCM.
8. In the **Repository URL** field, paste: `https://github.com/YOUR_USERNAME/Varun_Calculator.git`

9. **Credentials:** For a public repo, you can leave this as "- none -". If your repo is private, click **Add** → **Jenkins**, then add your GitHub username and personal access token.
10. In the **Branch Specifier** field, make sure it says `*/main`.
11. The **Script Path** should already say `Jenkinsfile` — that's correct. This tells Jenkins to look for a file named `Jenkinsfile` in the root of the repo.
12. Click **Save**.

Section 6 — The Jenkinsfile (Line-by-Line)

6.1 — What Is a Jenkinsfile?

A **Jenkinsfile** is a text file that defines your pipeline — the sequence of steps Jenkins should perform. It lives in your project's repository, so anyone can see how the project is built and deployed.

We'll use **Declarative Pipeline** syntax, which reads almost like English.

6.2 — Create the Jenkinsfile

Create a file called `Jenkinsfile` (no extension — just `Jenkinsfile`) in your `Varun_Calculator` folder:

> IMPORTANT
>
> The filename must be exactly `Jenkinsfile` with a capital `J` and no file extension. Not `Jenkinsfile.txt`, not `jenkinsfile`.

```groovy
// Jenkinsfile
// Declarative pipeline for the Python Calculator project.
// This file tells Jenkins exactly what to do when a build is triggered.

pipeline {

    // 'agent any' means: run this pipeline on any available Jenkins node (machine).
    agent any

    stages {

        // STAGE 1: Checkout - Get the latest code from GitHub
        stage('Checkout') {
            steps {
                // This pulls the latest code from the GitHub repository.
                // Jenkins does this automatically when using "Pipeline script from SCM",
                // but we include it here explicitly for clarity.
                checkout scm
            }
        }

        // STAGE 2: Install Dependencies
        // Our calculator uses only Python's built-in modules, so there's
        // nothing extra to install. In a real project, you'd install packages
        // from a requirements.txt file here.
        stage('Install Dependencies') {
            steps {
                echo 'No external dependencies to install.'
                echo 'This project uses only Python built-in modules.'
                // In a real project, you would run:
                // bat 'pip install -r requirements.txt'
            }
        }

        // STAGE 3: Run Tests - Automatically test the calculator
        stage('Run Tests') {
            steps {
                // Run the unittest test file.
                // 'bat' runs a Windows command. Use 'sh' instead on macOS/Linux.
                bat 'python -m unittest test_calculator.py -v'
            }
        }

        // STAGE 4: Build - Package the application
        stage('Build') {
```

```
            steps {
                echo 'Building the application...'
                // In a real Python project, you might create a distributable package.
                // For this project, we simulate packaging by echoing a message.
                echo 'Calculator application built successfully!'
            }
        }

        // STAGE 5: Deploy - Deliver the application
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
                // Simulate deployment: run the calculator with a sample input
                // to prove the application works end to end.
                bat 'echo 1 & echo 5 & echo 3 | python calculator.py'
                echo '----------------------------------------'
                echo 'Deployment simulation complete!'
                echo 'In a real environment, this stage would:'
                echo '  - Copy the app to a production server'
                echo '  - Restart the application service'
                echo '  - Notify the team via Slack or email'
                echo '----------------------------------------'
            }
        }
    }

    // POST section: runs after all stages complete
    post {
        success {
            echo '========================================'
            echo 'SUCCESS: All stages passed!'
            echo 'The Calculator app is tested and deployed.'
            echo '========================================'
        }
        failure {
            echo '========================================'
            echo 'FAILURE: Something went wrong!'
            echo 'Check the console output above for errors.'
            echo '========================================'
        }
    }
}
```

macOS / Linux Users

If you're on **macOS or Linux**, change every `bat` to `sh`:

```groovy
// Change this:
bat 'python -m unittest test_calculator.py -v'
// To this:
sh 'python3 -m unittest test_calculator.py -v'
```

And change:

```groovy
bat 'echo 1 & echo 5 & echo 3 | python calculator.py'
```

To:

```groovy
sh 'echo -e "1\\n5\\n3" | python3 calculator.py'
```

6.3 — Jenkinsfile Line-by-Line Explanation

Line / Block	What it does
`pipeline { }`	The outermost block — everything inside defines the pipeline.
`agent any`	Tells Jenkins to run the pipeline on any available machine. In bigger companies, you'd specify a specific server.
`stages { }`	Contains all the stages (steps) of the pipeline.
`stage('Checkout') { steps { checkout scm } }`	Downloads the latest code from GitHub. `scm` refers to the repository you configured in the job settings.
`stage('Install Dependencies')`	Where you'd install libraries your project needs. Ours doesn't need any.
`stage('Run Tests')`	Runs `python -m unittest test_calculator.py -v` — the same command you ran locally. If any test fails, **this stage fails, and the entire pipeline fails.**
`stage('Build')`	In a real project, this would compile code or create a package (like a `.zip` or `.whl` file). We use `echo` to simulate it.
`stage('Deploy')`	Simulates deploying the app. We pipe sample input into the calculator to prove it runs. In the real world, this would copy files to a server.
`post { success { ... } failure { ... } }`	Runs after all stages. If everything passed, print a success message. If anything failed, print a failure message.

6.4 — Push the Jenkinsfile to GitHub

Go back to your terminal:

```
bash
cd %USERPROFILE%\Desktop\Varun_Calculator
git add Jenkinsfile
git commit -m "Add Jenkinsfile for CI/CD pipeline"
git push
```

What you should see: Git uploads the Jenkinsfile to GitHub.

Go check your GitHub repo in the browser — you should now see three files: `calculator.py`, `test_calculator.py`, and `Jenkinsfile`.

6.5 — How to Make the Pipeline Fail on Purpose (For Your Presentation)

This is important for your demo — you want to show that the pipeline **catches bugs automatically**.

Step 1: Break a test deliberately

Open `calculator.py` and change the `add` function to return the **wrong** result:

```
python
def add(a, b):
    """Return the sum of a and b."""
    return a - b   # ← WRONG ON PURPOSE: minus instead of plus
```

Step 2: Push the broken code

```
bash
git add calculator.py
git commit -m "Introduce a bug to demonstrate pipeline failure"
git push
```

Step 3: Trigger a build in Jenkins

Go to Jenkins → click on **Calculator-Pipeline** → click **"Build Now"**.

Step 4: Watch it fail

Click on the build number (e.g., `#2`) → **Console Output**. You'll see something like:

```
test_add_positive_numbers (test_calculator.TestCalculator) ... FAIL

======================================================================
FAIL: test_add_positive_numbers (test_calculator.TestCalculator)
----------------------------------------------------------------------
AssertionError: -1 != 5

----------------------------------------------------------------------
Ran 10 tests in 0.001s

FAILED (failures=3)

...
FAILURE: Something went wrong!
Check the console output above for errors.
```

📸 **Screenshot this!** It proves the pipeline catches bugs.

The Jenkins dashboard will show the build with a **red circle** ❌ — meaning it failed.

Step 5: Fix the bug and push again

Change `calculator.py` back to the correct version:

```
python
def add(a, b):
    """Return the sum of a and b."""
    return a + b   # ← Fixed!
```

Push the fix:

```
bash
git add calculator.py
git commit -m "Fix the add function bug"
git push
```

Build again in Jenkins. This time it will pass ✅ with a **green circle** (or blue circle, depending on Jenkins theme).

📸 **Screenshot this too!** Now you have before (fail) and after (pass) screenshots for your presentation.

Section 7 — Running and Verifying

7.1 — Trigger a Build

There are two ways to trigger a build:

Manual (what we'll use):

1. In Jenkins, click on **Calculator-Pipeline**.
2. Click **"Build Now"** in the left sidebar.
3. A new build will appear under **"Build History"** (left sidebar, bottom).

Automatic (optional, for extra credit): You can configure Jenkins to automatically build when you push to GitHub by setting up a **webhook** — but that's beyond this project's scope.

7.2 — Read the Console Output

1. Click on the **build number** (e.g., `#1`) in the Build History.
2. Click **"Console Output"** in the left sidebar.
3. You'll see the complete log of everything Jenkins did:

```
Started by user admin
...
[Pipeline] stage (Checkout)
[Pipeline] checkout
  > git fetch ...
  > git checkout ...

[Pipeline] stage (Install Dependencies)
No external dependencies to install.

[Pipeline] stage (Run Tests)
[Calculator-Pipeline] Running bat script
> python -m unittest test_calculator.py -v
test_add_negative_numbers ... ok
test_add_positive_numbers ... ok
test_add_zero ... ok
test_divide_by_zero ... ok
test_divide_positive_numbers ... ok
test_divide_returns_float ... ok
test_multiply_by_zero ... ok
test_multiply_positive_numbers ... ok
test_subtract_negative_result ... ok
test_subtract_positive_numbers ... ok
----------------------------------------------------------------------
Ran 10 tests in 0.001s
OK

[Pipeline] stage (Build)
Calculator application built successfully!

[Pipeline] stage (Deploy)
Deploying the application...
============================
   SIMPLE CALCULATOR
============================
...

[Pipeline] echo
========================================
SUCCESS: All stages passed!
The Calculator app is tested and deployed.
========================================

Finished: SUCCESS
```

7.3 — Interpret the Results

What you see	What it means
`Finished: SUCCESS` with ✅ green/blue	All stages passed. Tests passed, build succeeded, deployment completed.
`Finished: FAILURE` with ❌ red	Something went wrong. Scroll up in Console Output to find the error.
`Finished: UNSTABLE` with ⚠️ yellow	Tests had warnings but didn't completely fail.

7.4 — The Pipeline Visualization

Back on the **Calculator-Pipeline** page, Jenkins shows a visual pipeline diagram:

```
Checkout → Install Dependencies → Run Tests → Build → Deploy
   ✅              ✅                ✅         ✅       ✅
```

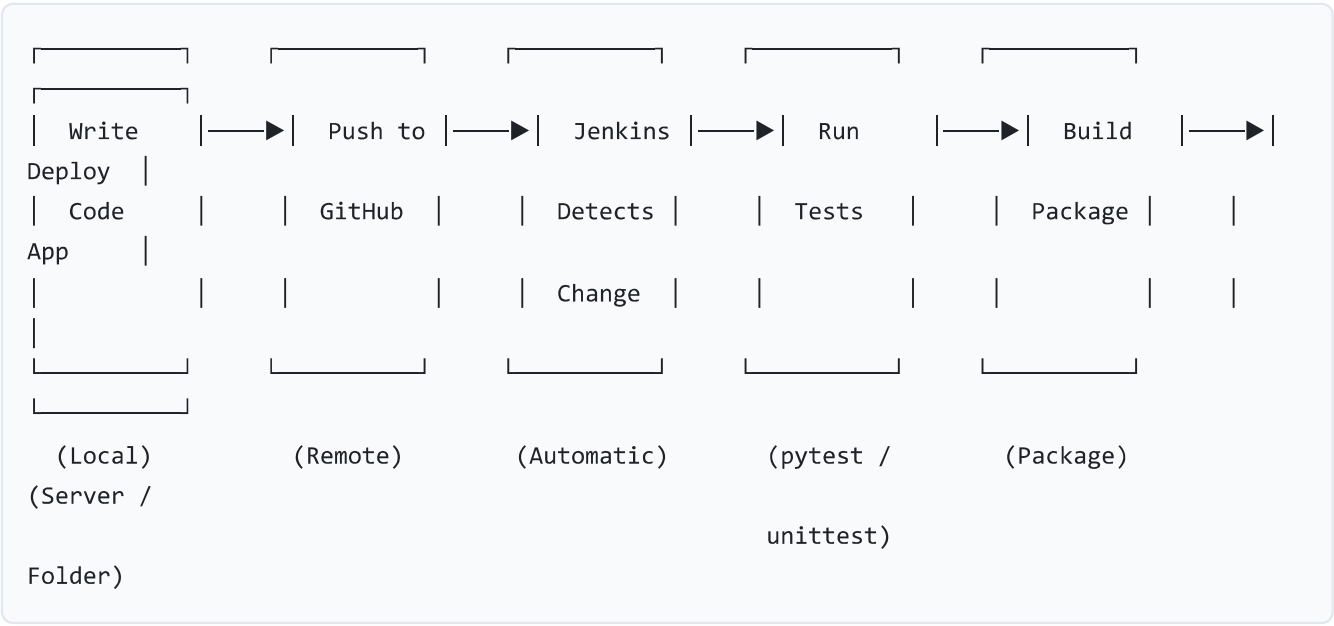
Each stage shows green (pass) or red (fail). This makes it instantly clear where a problem occurred.

📸 **Screenshot this pipeline visualization** — it's a great visual for your presentation.

Section 8 — Presentation Support

8.1 — Workflow Summary

Here's the complete CI/CD workflow you built, in one picture:



In words: "I write code on my computer, push it to GitHub, Jenkins automatically pulls the new code, runs tests to check for bugs, builds a deployable package, and deploys it. If any test fails, the pipeline stops and alerts me."

8.2 — Key Terms You Should Be Able to Define

Term	Definition
Python	A high-level programming language known for its readable syntax.
Git	A distributed version control system that tracks changes to files.
GitHub	A cloud-based platform that hosts Git repositories and enables collaboration.
Repository	A project folder tracked by Git, containing all code and its history.
Commit	A saved snapshot of changes with a descriptive message.
Push	Uploading local commits to a remote repository (like GitHub).
Branch	An independent line of development within a repository.
Jenkins	An open-source automation server used to build CI/CD pipelines.
CI (Continuous Integration)	The practice of automatically testing code every time it's pushed.
CD (Continuous Delivery)	The practice of automatically deploying code after it passes tests.
Pipeline	A defined sequence of automated stages (test, build, deploy).
Jenkinsfile	A text file in the repository that defines the Jenkins pipeline.
Stage	One step in a pipeline (e.g., "Run Tests" or "Deploy").
Unit Test	Code that automatically verifies a small piece of functionality works correctly.
Deployment	The process of making an application available for use (e.g., copying it to a server).

8.3 — Screenshot Checklist for Your Report

Use this checklist to capture everything you need for your project report:

- **GitHub repository page** showing all files (`calculator.py`, `test_calculator.py`, `Jenkinsfile`)
- **Calculator running locally** in the terminal (showing input and output)
- **Local tests passing** (output of `python -m unittest test_calculator.py -v`)
- **Jenkins Dashboard** showing the Calculator-Pipeline job
- **Pipeline Configuration** page (the settings where you connected GitHub)
- **Successful build — Console Output** (showing all tests passed, "SUCCESS")

- **Successful build — Pipeline Visualization** (the stage diagram with green boxes)
- **Failed build — Console Output** (showing the deliberately broken test, "FAILURE")
- **Failed build — Pipeline Visualization** (the stage diagram with a red box at "Run Tests")
- **Fixed build — Console Output** (showing the fix worked, "SUCCESS" again)
- **Git commit history** on GitHub (click "commits" on the repo page — shows all your commits)

8.4 — Suggested Presentation Flow (5–7 Minutes)

1. **Introduction (30 sec):** "I built a Python calculator and set up a CI/CD pipeline to automatically test and deploy it."
2. **Show the Calculator (1 min):** Run `python calculator.py` live. Show a calculation.
3. **Show the Tests (1 min):** Run `python -m unittest test_calculator.py -v` live. Point out all 10 tests passing.
4. **Explain GitHub (1 min):** Show the GitHub repo. Explain that this is where the code lives in the cloud.
5. **Explain the Pipeline (1 min):** Open Jenkins and show the pipeline visualization. Explain each stage briefly.
6. **Demo the Failure (1.5 min):** "Now I'll deliberately break the code to show how the pipeline catches bugs." Show the red/failed build. Then show the fix and the green/successful build.
7. **Conclusion (30 sec):** "This demonstrates how CI/CD works — code is automatically tested and deployed whenever changes are pushed. Bugs are caught early, saving time and preventing broken software from reaching users."

> **TIP**
>
> **Bonus tip for a strong impression:** During the Q&A, if someone asks "What would you do differently in a real project?", you can say: "*In a real project, I would add a* `requirements.txt` *for dependencies, use pytest instead of unittest for more readable tests, configure Jenkins webhooks for automatic triggers instead of clicking Build Now, deploy to a real server or cloud platform like AWS, and add stages for code quality checks using tools like flake8.*"

Congratulations! 🎉 You now have a complete, working CI/CD pipeline. You understand what each piece does, and you have everything you need for a great project presentation. Good luck!